

R Lab9. Algunos métodos estadísticos y de machine learning

Bioinformática. Ingeniería Biomédica

En bioinformática, el análisis estadístico de conjuntos de datos de diferentes tamaños y estructuras es un tarea muy frecuente. En este laboratorio vamos a centrarnos en algunas herramientas útiles tanto estadísticas como de *machine learning* que pueden resultar útiles y que no hemos visto en laboratorios anteriores.

Corregir p-valores para tener en cuenta el problema de comparaciones múltiples

En bioinformática frecuentemente realizamos miles de contrastes de hipótesis en un análisis lo que supone un grave problema por la aparición de muchos falsos positivos. Vamos a explorar algunas utilidades que existen implementadas en R para corregir los p-valores.

Supongamos que estamos trabajando con un conjunto de datos con 10000 genes y dos grupos de tamaño 10 cada uno de ellos.

1. Utiliza la función `rnorm()` para simular dos vectores de tamaño 10, `x` e `y`, y calcula el p-valor asociado a un contraste t-Student para dos muestras independientes con la función `t.test()`. Repite la simulación 10000 veces.

Solución:

Defino una función que simula una réplica de este experimento.

```
fun.random.ttest <-function(n1,n2){  
  # genero los dos vectores de tamaño n  
  x <- rnorm(n1)  
  y <- rnorm(n2)  
  # p-valor t.test  
  ttest.p <- t.test(x,y)$p.value  
  # devuelve el p.valor  
  return(ttest.p)  
}  
  
# simulo 1000 veces muestras de tamaño 10  
set.seed(1) # para replicar los resultados  
nrep<-10000  
pval<-rep(NA,nrep)  
  
for(k in 1:nrep)  
  pval[k]<-fun.random.ttest(n1=10,n2=10)
```

2. ¿Cuántos p-valores son significativos a nivel 0.05?

Solución:

```
sum(pval<=0.05)
```

```
## [1] 506
```

3. Ajustamos los p-valores utilizando la función `p.adjust()`. Esta función tiene como primer argumento el vector de p-valores sin ajustar y en el argumento `method` le debéis indicar el método a utilizar para realizar el ajuste. Devuelve el vector de los p-valores ajustados.

Utiliza el método de Benjamini & Hochberg para realizar el ajuste por comparaciones múltiples. ¿Cuántos contrastes dejan de ser estadísticamente significativos? ¿Es lo esperado? Justifica la respuesta.

Consulta la ayuda de la función `p.adjust()`, `help("p.adjust")`, donde está disponible toda la información sobre los métodos implementados.

Solución:

Para utilizar el método de Benjamini & Hochberg se corresponde debemos utilizar el argumento `method="BH"`.

```
adj.pval.BH <- p.adjust(pval, method = "BH")  
  
sum(adj.pval.BH<=0.05)
```

```
## [1] 0
```

En este caso, no debería haber ningún p-valor estadísticamente significativo, puesto que tanto `x` como `y` se han generado utilizando la misma distribución, sin embargo, en los p-valores sin ajustar tenemos 506 (5.06%) falsos positivos. El método de ajuste sirve para corregir los p-valores.

Predecir clases con diferentes algoritmos

En esta sección vamos a ver distintos algoritmos para resolver el problema de predecir la clase a la que pertenece un individuo. Son algoritmos de aprendizaje supervisado en el que a partir de un conjunto de datos establecemos un modelo que nos permite predecir la clase a la que pertenecería un nuevo individuo del que conocemos esa información.

Predecir clases con el algoritmo de los k-vecinos más próximos (kNN)

El algoritmo de los k vecinos más próximos (*k-Nearest Neighbors*: kNN) trata de clasificar los individuos de un conjunto de datos basándose en la similitud de una observación con k de las observaciones más cercanas. Es un método supervisado en el que se conoce la clase a la que pertenece cada individuo.

En esta sección vamos a dividir un conjunto de datos en dos partes: conjunto de entrenamiento (*train set*) y un conjunto de validación (*test set*). Trataremos de predecir la clase a la que pertenece cada individuo del *test set* a partir del modelo construido con el *train set*.

Utilizamos el conjunto de datos `iris` que contiene información de flores de iris de 3 especies diferentes: iris setosa, versicolor y virginica. Concretamente contiene 150 observaciones y 5 variables: longitud y anchura del sépalo, longitud y anchura del pétalo y la especie. Esta disponible en el paquete `datasets` y podéis cargarlo con la instrucción:

```
data(iris)
head(iris)
```

```
## Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1      5.1      3.5      1.4      0.2 setosa
## 2      4.9      3.0      1.4      0.2 setosa
## 3      4.7      3.2      1.3      0.2 setosa
## 4      4.6      3.1      1.5      0.2 setosa
## 5      5.0      3.6      1.4      0.2 setosa
## 6      5.4      3.9      1.7      0.4 setosa
```

1. Construye un nuevo `data.frame`, llamado `scaled.iris`, que contenga las variables numéricas estandarizadas. Para estandarizar una variable podéis utilizar la función genérica `scale()`.

Partir de los datos estandarizados es muy importante en el algoritmo kNN, ya que valores de mayores magnitudes tendrán un efecto mayor en el modelo resultado, por lo que necesitamos que las variables sean comparables.

Solución:

Podemos utilizar la función `sapply()`,

```
scaled.iris <- sapply(1:4,function(i) scale(iris[,i]))
colnames(scaled.iris) <- colnames(iris)[1:4]
head(scaled.iris)
```

```
## Sepal.Length Sepal.Width Petal.Length Petal.Width
## [1,] -0.8976739 1.01560199 -1.335752 -1.311052
## [2,] -1.1392005 -0.13153881 -1.335752 -1.311052
## [3,] -1.3807271 0.32731751 -1.392399 -1.311052
## [4,] -1.5014904 0.09788935 -1.279104 -1.311052
## [5,] -1.0184372 1.24503015 -1.335752 -1.311052
## [6,] -0.5353840 1.93331463 -1.165809 -1.048667
```

2. Utiliza la función `sample()` para dividir la muestra en dos grupos: *train* y *test set*. El conjunto de entrenamiento debe contener el 80% de las observaciones. Recuerda utilizar `set.seed()` para poder reproducir los resultados.

Solución:

```
set.seed(123)
train.rows <- sample(nrow(scaled.iris), 0.8 * nrow(scaled.iris), replace = FALSE)
train.set <- scaled.iris[train.rows, ]
test.set <- scaled.iris[-train.rows, ]
```

3. Construye el modelo con el *train set*. El algoritmo kNN está implementado en la función `knn()` del paquete `class` disponible en CRAN.

```
if(!("class" %in% installed.packages()))
  install.packages("class")
library("class")
```

Los argumentos importantes de la función `knn()` son: `train` en el que debéis pasarle el *train set* (sólo las variables numéricas), `test` para pasarle el *test set*, `cl` un vector con la información de las clases del *train set* y `k` en el que hay que indicar el número de vecinos a considerar (probar con 10). La función devuelve un vector con las clases predichas para el *test set*.

Solución:

```
test.set.predictions <- knn(train = train.set, test = test.set,  
                             cl = iris[train.rows,"Species"], k = 10)  
test.set.predictions
```

```
## [1] setosa  setosa  setosa  setosa  setosa  setosa  
## [7] setosa  setosa  setosa  setosa  versicolor versicolor  
## [13] versicolor versicolor versicolor versicolor versicolor versicolor  
## [19] versicolor versicolor versicolor versicolor virginica  versicolor  
## [25] versicolor virginica  virginica  virginica  virginica  virginica  
## Levels: setosa versicolor virginica
```

4. Compara los resultados de la predicción con los valores reales. Podrías construir una tabla de contingencia que relacione las dos variables. En el paquete `caret` disponible en CRAN está implementada la función `confusionMatrix()`, que además de esa tabla calcula estadísticos sobre la precisión de la predicción. Interpreta los resultados obtenidos.

```
if(!("caret" %in% installed.packages()))  
  install.packages("caret")  
library("caret")
```

Solución:

```
caret::confusionMatrix(test.set.predictions, iris[-train.rows,"Species"])
```

```
## Confusion Matrix and Statistics  
##  
##           Reference  
## Prediction  setosa versicolor virginica  
## setosa      10      0      0  
## versicolor   0     14      0  
## virginica    0      1      5  
##  
## Overall Statistics  
##  
##           Accuracy : 0.9667  
##           95% CI : (0.8278, 0.9992)  
##    No Information Rate : 0.5  
##    P-Value [Acc > NIR] : 2.887e-08  
##  
##           Kappa : 0.9464  
##  
## Mcnemar's Test P-Value : NA  
##  
## Statistics by Class:  
##  
##           Class: setosa Class: versicolor Class: virginica  
## Sensitivity           1.0000           0.9333           1.0000  
## Specificity           1.0000           1.0000           0.9600  
## Pos Pred Value         1.0000           1.0000           0.8333  
## Neg Pred Value         1.0000           0.9375           1.0000  
## Prevalence             0.3333           0.5000           0.1667  
## Detection Rate         0.3333           0.4667           0.1667  
## Detection Prevalence   0.3333           0.4667           0.2000  
## Balanced Accuracy      1.0000           0.9667           0.9800
```

Se predice mal una flor de la especie versicolor que se clasifica como virginica. La precisión es del 96.67% con un intervalo de confianza entre 82.78% y 99.92%. También tenemos los valores de la sensibilidad y especificidad, junto con los valores predictivos, positivo y negativo.

Predecir clases con *random forest*

Random forest es un algoritmo de aprendizaje supervisado que utiliza un conjunto de árboles de decisión para hacer predicciones sobre a que clase pertenece un individuo. Podemos trabajar con variables tanto cuantitativas como cualitativas y se aplica tanto a problemas de clasificación como de regresión. Como veremos después, también se puede utilizar para determinar que variables son más relevantes en un conjunto de datos.

En esta sección vamos a utilizar *random forest* para predecir las clases a las que pertenecen los individuos. Utilizamos el paquete `randomForest` disponible en CRAN y los datos `iris` ya utilizados en el apartado anterior.

```
if(!("randomForest" %in% installed.packages()))  
  install.packages("randomForest")  
library("randomForest")
```

```
data(iris)  
head(iris)
```

```
## Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1      5.1      3.5      1.4      0.2 setosa
## 2      4.9      3.0      1.4      0.2 setosa
## 3      4.7      3.2      1.3      0.2 setosa
## 4      4.6      3.1      1.5      0.2 setosa
## 5      5.0      3.6      1.4      0.2 setosa
## 6      5.4      3.9      1.7      0.4 setosa
```

1. Utiliza la función `sample()` para dividir la muestra en los dos grupos definidos en el punto 2 de la sección anterior. En este caso no es necesario estandarizar los datos.

Solución:

```
set.seed(123)
train.rows <- sample(nrow(iris), 0.8 * nrow(iris), replace = FALSE)
train.set <- iris[train.rows, ]
test.set <- iris[-train.rows, ]
```

2. Utiliza la función `randomForest()` para construir el modelo con el *train set*. El primer argumento de esta función es una fórmula del tipo `Species ~ .`. En la parte izquierda se indica la variable que clasifica a los individuos. En la parte derecha debemos poner las variables utilizadas para hacer la predicción. El `.` indica que todas las variables salvo `Species`. El segundo argumento, `data`, es es `data.frame` en el que se basa el ajuste.

Solución:

```
model <- randomForest(Species ~ ., data = train.set)
model
```

```
##
## Call:
## randomForest(formula = Species ~ ., data = train.set)
##           Type of random forest: classification
##           Number of trees: 500
## No. of variables tried at each split: 2
##
## OOB estimate of error rate: 4.17%
## Confusion matrix:
##      setosa versicolor virginica class.error
## setosa      40         0         0 0.00000000
## versicolor   0        32         3 0.08571429
## virginica    0         2        43 0.04444444
```

Notad que se obtiene una matriz de confusión que se refiere a los mismos datos que se han utilizado para ajustar el modelo.

3. Utiliza la función `predict()` para hacer las predicciones en el *test set*. Los argumentos a utilizar son: el modelo ajustado en 2, el `data.frame` que contiene los nuevos datos y `type="class"`, para indicarle que la predicción debe ser la clase.

Solución:

```
test.predictions <- predict(model, test.set, type = "class")
test.predictions
```

```
##      1      2      3      11      18      19      28
## setosa setosa setosa setosa setosa setosa setosa
## 33     36     48     55     56     57     58
## setosa setosa setosa versicolor versicolor versicolor versicolor
## 59     61     62     65     66     70     77
## versicolor versicolor versicolor versicolor versicolor versicolor versicolor
## 83     84     98     100     105     113     125
## versicolor virginica versicolor versicolor virginica virginica virginica
## 131     141
## virginica virginica
## Levels: setosa versicolor virginica
```

4. Compara los resultados de la predicción con los valores reales.

Solución:

```
caret::confusionMatrix(test.predictions, test.set$Species)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  setosa versicolor virginica
## setosa      10      0      0
## versicolor   0     14      0
## virginica    0      1      5
##
## Overall Statistics
##
##           Accuracy : 0.9667
##           95% CI : (0.8278, 0.9992)
##   No Information Rate : 0.5
##   P-Value [Acc > NIR] : 2.887e-08
##
##           Kappa : 0.9464
##
## Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##           Class: setosa Class: versicolor Class: virginica
## Sensitivity           1.0000           0.9333           1.0000
## Specificity           1.0000           1.0000           0.9600
## Pos Pred Value        1.0000           1.0000           0.8333
## Neg Pred Value        1.0000           0.9375           1.0000
## Prevalence            0.3333           0.5000           0.1667
## Detection Rate        0.3333           0.4667           0.1667
## Detection Prevalence  0.3333           0.4667           0.2000
## Balanced Accuracy      1.0000           0.9667           0.9800
```

Se predice mal una flor de la especie versicolor que se clasifica como virginica. La precisión es del 96.67% con un intervalo de confianza entre 82.78% y 99.92%. Obtenemos también los valores de sensibilidad, especificidad junto con los valores predictivos.

Predecir clases con SVM

El algoritmo *support vector machine* (SVM) es un clasificador que se basa en encontrar la diferencia máxima entre clases en múltiples dimensiones de los datos.

Vamos a utilizar SVM para realizar la predicción de clases e ilustrar el procedimiento gráficamente. Utilizamos el paquete `e1071` disponible en CRAN y los datos `iris` ya utilizados en los apartados anteriores.

```
if(!("e1071" %in% installed.packages()))
  install.packages("e1071")
library("e1071")

data(iris)
head(iris)
```

```
## Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1      5.1      3.5      1.4      0.2 setosa
## 2      4.9      3.0      1.4      0.2 setosa
## 3      4.7      3.2      1.3      0.2 setosa
## 4      4.6      3.1      1.5      0.2 setosa
## 5      5.0      3.6      1.4      0.2 setosa
## 6      5.4      3.9      1.7      0.4 setosa
```

1. Utiliza la función `sample()` para dividir la muestra en los dos grupos definidos en los apartados anteriores. En este caso tampoco es necesario estandarizar los datos.

Solución:

```
set.seed(123)
train.rows <- sample(nrow(iris), 0.8 * nrow(iris), replace = FALSE)
train.set <- iris[train.rows, ]
test.set <- iris[-train.rows, ]
```

2. Utiliza la función `svm()` para construir el modelo con el *train set*.

- El primer argumento de esta función es una fórmula del tipo `Especie ~ .`. En la parte izquierda se indica la variable que clasifica a los individuos. En la parte derecha debemos poner las variables utilizadas para hacer la predicción. El `.` indica que todas las variables salvo `Especie`.
- El segundo argumento, `data`, es el `data.frame` en el que se basa el ajuste.
- El argumento `type` sirve para indicar si el objetivo del ajuste es la clasificación o la regresión. El valor de clasificación más utilizado es

type="C-classification" .

- En el argumento `kernel` indicamos la función a utilizar para ajustar el modelo. Puede ser: `linear` , `polynomial` , `radial` o `sigmoid` . La mejor elección depende de los datos. Una estrategia es probar con varios y quedarnos con el mejor según algún criterio, por ejemplo el que menor error de predicción cometa. En este caso utilizamos `kernel="radial"` . La expresión de la función kernel radial es $\exp(-\gamma * |u - v|^2)$.
- `gamma` es un parámetro del kernel. Fijarlo en `0.25`.

Solución:

```
model <- svm(Species~., data=train.set, type="C-classification", kernel="radial", gamma=0.25)
model
```

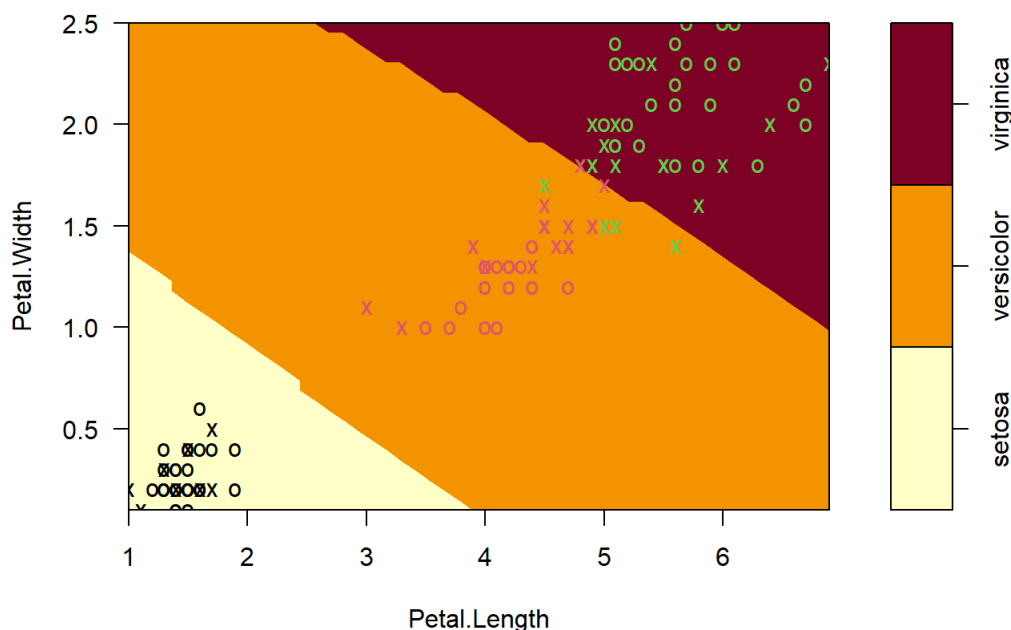
```
##
## Call:
## svm(formula = Species ~ ., data = train.set, type = "C-classification",
##      kernel = "radial", gamma = 0.25)
##
##
## Parameters:
##  SVM-Type: C-classification
##  SVM-Kernel: radial
##      cost: 1
##
## Number of Support Vectors: 46
```

3. Representa gráficamente el modelo con el método `plot()` . Utiliza el siguiente código e interpreta el gráfico resultante.

```
cols.to.hold <- c("Sepal.Length", "Sepal.Width")
held.constant <- lapply(cols.to.hold, function(x){mean(train.set[[x]])})
names(held.constant) <- cols.to.hold
plot(model, train.set, Petal.Width ~ Petal.Length, slice =held.constant)
```

Solución:

SVM classification plot



En este gráfico los puntos representados por 'X' son observaciones que influyen en el ajuste mientras que los representados con 'O' son los que no influyen en este ajuste. El color de los puntos representa la clase.

4. Utiliza la función `predict()` para hacer las predicciones en el test set. Los argumentos a utilizar son: el modelo ajustado en 2, el `data.frame` que contiene los nuevos datos y `type="class"` , para indicarle que la predicción debe ser la clase.

Solución:

```
test.predictions <- predict(model, test.set, type = "class")
test.predictions
```

```
##      1      2      3      11      18      19      28
## setosa setosa setosa setosa setosa setosa setosa
##    33    36    48    55    56    57    58
## setosa setosa setosa versicolor versicolor versicolor versicolor
##    59    61    62    65    66    70    77
## versicolor versicolor versicolor versicolor versicolor versicolor versicolor
##    83    84    98   100   105   113   125
## versicolor virginica versicolor versicolor virginica virginica virginica
##   131   141
## virginica virginica
## Levels: setosa versicolor virginica
```

5. Compara los resultados de la predicción con los valores reales.

Solución:

```
caret::confusionMatrix(test.predictions, test.set$Species)
```

```
## Confusion Matrix and Statistics
##
##      Reference
## Prediction setosa versicolor virginica
## setosa      10      0      0
## versicolor   0     14      0
## virginica    0      1      5
##
## Overall Statistics
##
##      Accuracy : 0.9667
##      95% CI : (0.8278, 0.9992)
##      No Information Rate : 0.5
##      P-Value [Acc > NIR] : 2.887e-08
##
##      Kappa : 0.9464
##
##      Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##      Class: setosa Class: versicolor Class: virginica
## Sensitivity        1.0000         0.9333         1.0000
## Specificity         1.0000         1.0000         0.9600
## Pos Pred Value       1.0000         1.0000         0.8333
## Neg Pred Value       1.0000         0.9375         1.0000
## Prevalence           0.3333         0.5000         0.1667
## Detection Rate       0.3333         0.4667         0.1667
## Detection Prevalence 0.3333         0.4667         0.2000
## Balanced Accuracy     1.0000         0.9667         0.9800
```

Se predice mal una flor de la especie versicolor que se clasifica como virginica. La precisión es del 96.67% con un intervalo de confianza entre 82.78% y 99.92%. Obtenemos también los valores de sensibilidad, especificidad junto con los valores predictivos.

Agrupar observaciones sin información previa

En bioinformática es muy común querer clasificar observaciones en grupos sin tener ninguna información previa sobre cuantos grupos o como deben ser estos grupos. Queremos hacer *clustering*, un tipo de aprendizaje no supervisado.

En esta sección vamos a comenzar con un conjunto de datos de expresión de 150 muestras, vamos a averiguar cuantos grupos de muestras deberíamos hacer y aplicar un método de *clustering* basado en reducir la dimensionalidad con un análisis de componentes principales seguido del algoritmo de las k medias.

Vamos a utilizar los paquetes `factoextra` y `Biobase` disponibles en CRAN y Bioconductor, respectivamente, y el fichero `modencodefly_eset.RData` disponible en el campus. La extensión `RDATA` significa *R Workspace Format* y almacena los objetos creados en el espacio de trabajo. Para cargar los datos utilizamos la función `load()`.

```
if(!("factoextra" %in% installed.packages()))
  install.packages("factoextra")

if(!("Biobase" %in% installed.packages()))
  BiocManager::install("Biobase")

library("factoextra")
library("Biobase")

load(file.path("datos", "modencodefly_eset.RData"))
```

1. Utiliza la función `prcomp()` para hacer de la matriz de expresión del objeto `ExpressionSet` `modencodefly.eset` obtenida con la función `exprs()`. Utiliza los argumentos `scale = TRUE` y `center = TRUE` para estandarizar la matriz de expresión. ¿Cuál es la variabilidad total explicada por la primera componente?

Solución:

```
expr.pca <- prcomp(exprs(modencodefly.eset), scale=TRUE, center=TRUE )
summary(expr.pca)
```

```
## Importance of components:
##          PC1  PC2  PC3  PC4  PC5  PC6  PC7
## Standard deviation   8.8437 3.8891 3.62546 3.0600 2.47080 2.14908 1.94841
## Proportion of Variance 0.5321 0.1029 0.08941 0.0637 0.04153 0.03142 0.02583
## Cumulative Proportion 0.5321 0.6349 0.72435 0.7881 0.82958 0.86100 0.88683
##          PC8  PC9  PC10  PC11  PC12  PC13  PC14
## Standard deviation   1.79604 1.76973 1.3228 1.20585 1.1180 1.05226 0.97912
## Proportion of Variance 0.02194 0.02131 0.0119 0.00989 0.0085 0.00753 0.00652
## Cumulative Proportion 0.90877 0.93008 0.9420 0.95187 0.9604 0.96791 0.97443
##          PC15  PC16  PC17  PC18  PC19  PC20  PC21
## Standard deviation   0.82974 0.76919 0.70935 0.63903 0.57150 0.47064 0.44440
## Proportion of Variance 0.00468 0.00402 0.00342 0.00278 0.00222 0.00151 0.00134
## Cumulative Proportion 0.97911 0.98314 0.98656 0.98934 0.99156 0.99307 0.99441
##          PC22  PC23  PC24  PC25  PC26  PC27  PC28
## Standard deviation   0.43454 0.40588 0.35883 0.27545 0.25293 0.21189 0.18111
## Proportion of Variance 0.00128 0.00112 0.00088 0.00052 0.00044 0.00031 0.00022
## Cumulative Proportion 0.99569 0.99682 0.99769 0.99821 0.99864 0.99895 0.99917
##          PC29  PC30  PC31  PC32  PC33  PC34  PC35
## Standard deviation   0.15062 0.13840 0.1238 0.10006 0.08937 0.07025 0.06712
## Proportion of Variance 0.00015 0.00013 0.0001 0.00007 0.00005 0.00003 0.00003
## Cumulative Proportion 0.99933 0.99946 0.9996 0.99963 0.99968 0.99972 0.99975
##          PC36  PC37  PC38  PC39  PC40  PC41  PC42
## Standard deviation   0.06129 0.05693 0.05478 0.04812 0.04506 0.03670 0.03482
## Proportion of Variance 0.00003 0.00002 0.00002 0.00002 0.00001 0.00001 0.00001
## Cumulative Proportion 0.99977 0.99979 0.99981 0.99983 0.99984 0.99985 0.99986
##          PC43  PC44  PC45  PC46  PC47  PC48  PC49
## Standard deviation   0.03393 0.03162 0.03044 0.02897 0.02839 0.02741 0.02643
## Proportion of Variance 0.00001 0.00001 0.00001 0.00001 0.00001 0.00001 0.00000
## Cumulative Proportion 0.99987 0.99988 0.99988 0.99989 0.99989 0.99990 0.99990
##          PC50  PC51  PC52  PC53  PC54  PC55  PC56
## Standard deviation   0.02564 0.02464 0.02402 0.02274 0.02238 0.02127 0.01996
## Proportion of Variance 0.00000 0.00000 0.00000 0.00000 0.00000 0.00000 0.00000
## Cumulative Proportion 0.99991 0.99991 0.99992 0.99992 0.99992 0.99993 0.99993
##          PC57  PC58  PC59  PC60  PC61  PC62  PC63
## Standard deviation   0.01988 0.01961 0.01837 0.01796 0.0176 0.01677 0.01674
## Proportion of Variance 0.00000 0.00000 0.00000 0.00000 0.0000 0.00000 0.00000
## Cumulative Proportion 0.99993 0.99993 0.99994 0.99994 0.9999 0.99994 0.99994
##          PC64  PC65  PC66  PC67  PC68  PC69  PC70
## Standard deviation   0.01609 0.01575 0.01562 0.01524 0.01481 0.01453 0.01434
## Proportion of Variance 0.00000 0.00000 0.00000 0.00000 0.00000 0.00000 0.00000
## Cumulative Proportion 0.99995 0.99995 0.99995 0.99995 0.99995 0.99995 0.99996
##          PC71  PC72  PC73  PC74  PC75  PC76  PC77
## Standard deviation   0.01418 0.01384 0.01369 0.01347 0.01332 0.01319 0.01291
## Proportion of Variance 0.00000 0.00000 0.00000 0.00000 0.00000 0.00000 0.00000
## Cumulative Proportion 0.99996 0.99996 0.99996 0.99996 0.99996 0.99996 0.99996
##          PC78  PC79  PC80  PC81  PC82  PC83  PC84
## Standard deviation   0.01267 0.01249 0.01216 0.012 0.01185 0.01172 0.01157
## Proportion of Variance 0.00000 0.00000 0.00000 0.000 0.00000 0.00000 0.00000
## Cumulative Proportion 0.99997 0.99997 0.99997 1.000 0.99997 0.99997 0.99997
##          PC85  PC86  PC87  PC88  PC89  PC90  PC91
## Standard deviation   0.01153 0.01131 0.01119 0.01095 0.01093 0.0108 0.01056
## Proportion of Variance 0.00000 0.00000 0.00000 0.00000 0.00000 0.0000 0.00000
## Cumulative Proportion 0.99997 0.99997 0.99997 0.99997 0.99997 0.99998 1.0000 0.99998
##          PC92  PC93  PC94  PC95  PC96  PC97
## Standard deviation   0.01045 0.01037 0.01023 0.009922 0.009859 0.009764
## Proportion of Variance 0.00000 0.00000 0.00000 0.000000 0.000000 0.000000
## Cumulative Proportion 0.99998 0.99998 0.99998 0.999980 0.999980 0.999980
##          PC98  PC99  PC100  PC101  PC102  PC103
## Standard deviation   0.009679 0.009576 0.009526 0.009427 0.009252 0.009176
## Proportion of Variance 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000
## Cumulative Proportion 0.999980 0.999980 0.999980 0.999980 0.999980 0.999980
##          PC104  PC105  PC106  PC107  PC108  PC109
## Standard deviation   0.009085 0.008949 0.008866 0.008642 0.008627 0.008583
## Proportion of Variance 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000
## Cumulative Proportion 0.999990 0.999990 0.999990 0.999990 0.999990 0.999990
##          PC110  PC111  PC112  PC113  PC114  PC115
## Standard deviation   0.008453 0.00839 0.008294 0.008181 0.008121 0.008047
## Proportion of Variance 0.000000 0.00000 0.000000 0.000000 0.000000 0.000000
```



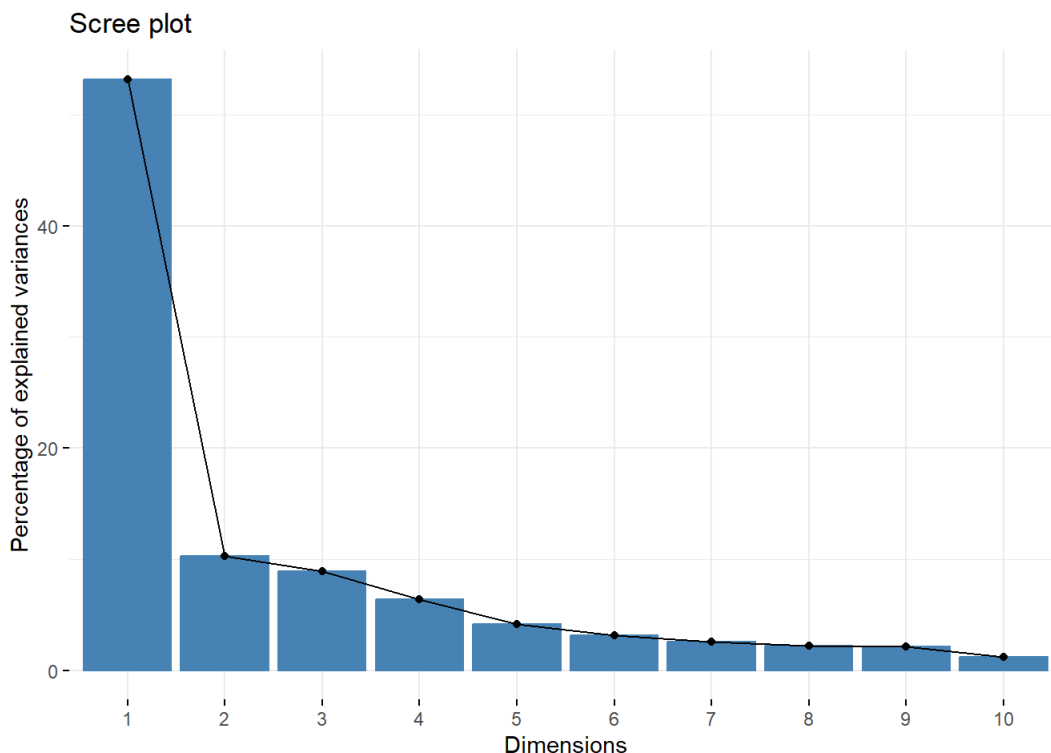
```
## Cumulative Proportion 0.999990 0.999990 0.999990 0.999990 0.999990 0.999990
## PC116 PC117 PC118 PC119 PC120 PC121
## Standard deviation 0.007998 0.007885 0.007866 0.007774 0.007659 0.007567
## Proportion of Variance 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000
## Cumulative Proportion 0.999990 0.999990 0.999990 0.999990 0.999990 0.999990
## PC122 PC123 PC124 PC125 PC126 PC127
## Standard deviation 0.007523 0.007289 0.007194 0.007135 0.007056 0.0069
## Proportion of Variance 0.000000 0.000000 0.000000 0.000000 0.000000 0.0000
## Cumulative Proportion 0.999990 0.999990 0.999990 0.999990 1.000000 1.0000
## PC128 PC129 PC130 PC131 PC132 PC133
## Standard deviation 0.006812 0.006734 0.006645 0.006583 0.006517 0.006448
## Proportion of Variance 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000
## Cumulative Proportion 1.000000 1.000000 1.000000 1.000000 1.000000 1.000000
## PC134 PC135 PC136 PC137 PC138 PC139
## Standard deviation 0.006363 0.006274 0.006091 0.006082 0.006027 0.005889
## Proportion of Variance 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000
## Cumulative Proportion 1.000000 1.000000 1.000000 1.000000 1.000000 1.000000
## PC140 PC141 PC142 PC143 PC144 PC145
## Standard deviation 0.005763 0.005532 0.005207 0.005125 0.00472 0.004547
## Proportion of Variance 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000
## Cumulative Proportion 1.000000 1.000000 1.000000 1.000000 1.000000 1.000000
## PC146 PC147
## Standard deviation 0.004318 0.003747
## Proportion of Variance 0.000000 0.000000
## Cumulative Proportion 1.000000 1.000000
```

La primera componente explica el 53.21% de la variabilidad total de la muestra.

2. Utiliza la función `fviz_screplot()` de `factoextra` para representar el *scree plot* del objeto PCA. Este plot es similar a un diagrama de barras en el que se representa la variabilidad explicada por cada componente. ¿Cuántas componentes resumirían los datos capturando al menos el 70% de su variabilidad?

Solución:

```
fviz_screplot(expr.pca)
```



Entre las tres primeras componentes recogen más del 70% de la variabilidad, con lo que vamos a utilizar estas tres componentes en lugar de las 150 variables originales.

3. La función `prcomp()` que realiza en PCA devuelve una lista. El elemento `rotation` de esa lista contiene las componentes principales. Selecciona el número de componentes que recogen al menos el 70% de la variabilidad de los datos.

Solución:

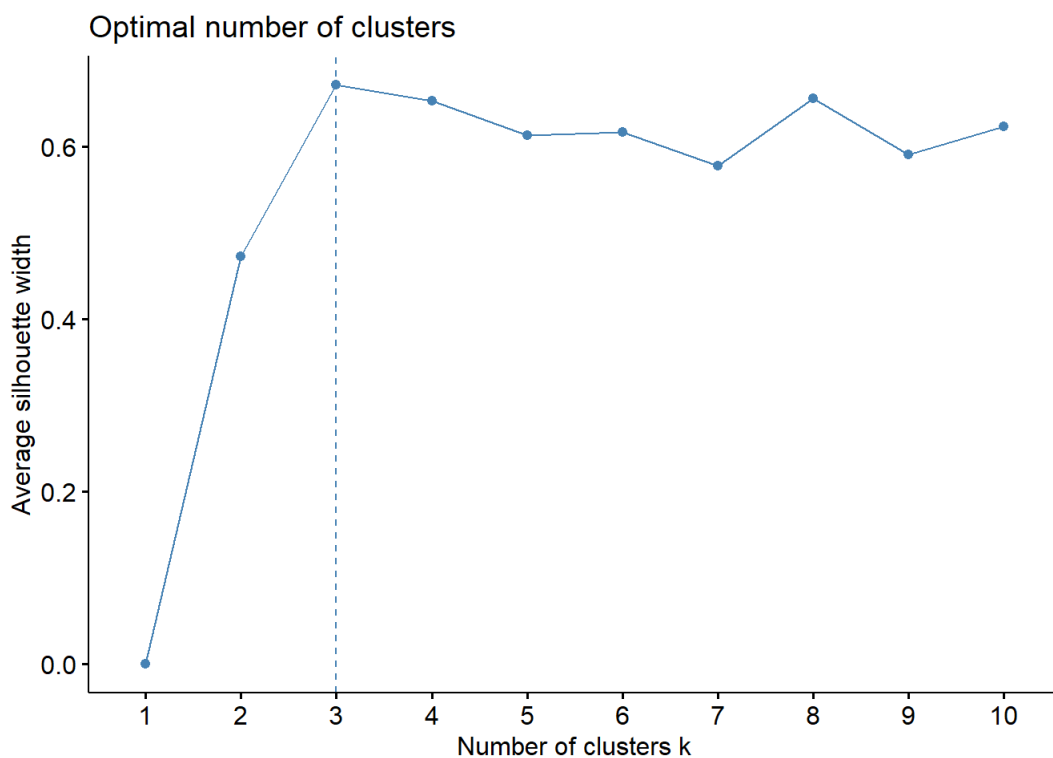
```
main.components <- expr.pca$rotation[, 1:3]
head(main.components)
```

```
##          PC1      PC2      PC3
## SRX007811 0.09140349 -0.06958202 -0.06572828
## SRX008180 0.09109687 -0.07021383 -0.06546600
## SRX008227 0.09126505 -0.06976318 -0.06565140
## SRX008238 0.09107530 -0.07001204 -0.06557046
## SRX008258 0.09097037 -0.07032466 -0.06534883
## SRX008015 0.09653502 -0.08260890 -0.07062517
```

4. Utiliza la función `fviz_nbclust()` para determinar cuál es el número óptimo de clusters a tener en cuenta con el algoritmo de las k medias. Esta función requiere como argumento el `data.frame` obtenido en 3. Además hay que pasarle el argumento `FUNcluster = kmeans` para indicarle que se quiere obtener el número óptimo de clusters con el algoritmo de las k medias. ¿Cuántos clusters son los óptimos en este caso?

Solución:

```
fviz_nbclust(main.components, FUNcluster = kmeans)
```



El número de clusters óptimos es 3.

5. Utiliza la función `kmeans()` para llevar a cabo el *clustering*. Además de los datos de las componentes principales a agrupar, es necesario pasarle el número de clusters a formar en el argumento `centers`.

Solución:

```
kmean.clus <- kmeans(main.components, centers= 3)
kmean.clus
```

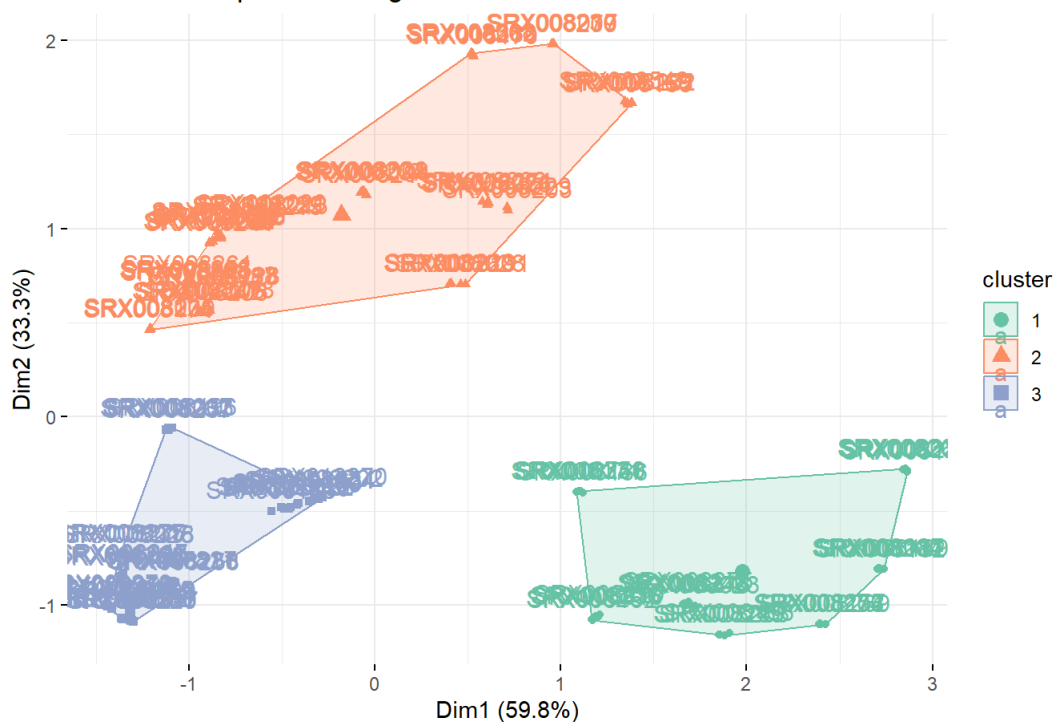
```
## K-means clustering with 3 clusters of sizes 34, 62, 51
##
## Cluster means:
##      PC1      PC2      PC3
## 1 0.05570626 -0.07297403 0.13637808
## 2 0.08382814 0.08825642 0.01317872
## 3 0.09219042 -0.06037450 -0.06273498
##
## Clustering vector:
## SRX007811 SRX008180 SRX008227 SRX008238 SRX008258 SRX008015 SRX008179 SRX008190
##      3      3      3      3      3      3      3      3
## SRX008193 SRX008271 SRX008027 SRX008181 SRX008217 SRX008250 SRX008265 SRX008025
##      3      3      3      3      3      3      3      3
## SRX008175 SRX008210 SRX008212 SRX008257 SRX008010 SRX008249 SRX008252 SRX008273
##      3      3      3      3      3      3      3      3
## SRX008274 SRX008005 SRX008198 SRX008208 SRX008243 SRX008247 SRX008018 SRX008177
##      3      3      3      3      3      3      3      3
## SRX008225 SRX008235 SRX008277 SRX008007 SRX008196 SRX008233 SRX008237 SRX008262
##      3      3      3      3      3      3      3      3
## SRX008006 SRX008178 SRX008205 SRX008242 SRX008278 SRX008020 SRX008213 SRX008215
##      2      2      2      2      2      2      2      2
## SRX008222 SRX008259 SRX008011 SRX008214 SRX008221 SRX008241 SRX008256 SRX008019
##      2      2      2      2      2      2      2      2
## SRX008167 SRX008234 SRX008251 SRX008266 SRX008026 SRX008174 SRX008201 SRX008239
##      2      2      2      2      2      2      2      2
## SRX008008 SRX008168 SRX008211 SRX008255 SRX008261 SRX008009 SRX008194 SRX008207
##      2      2      2      2      2      2      2      2
## SRX008224 SRX008244 SRX008029 SRX008184 SRX008206 SRX008220 SRX008267 SRX008014
##      2      2      1      1      1      1      1      1
## SRX008182 SRX008187 SRX008189 SRX008200 SRX008156 SRX008199 SRX008245 SRX008253
##      1      1      1      1      1      1      1      1
## SRX008023 SRX008218 SRX008246 SRX008248 SRX008275 SRX008016 SRX008192 SRX008219
##      1      1      1      1      1      1      1      1
## SRX008236 SRX008264 SRX008013 SRX008188 SRX008197 SRX008228 SRX008022 SRX008183
##      1      1      2      2      2      2      3      3
## SRX008185 SRX008216 SRX012269 SRX008024 SRX008173 SRX008195 SRX008202 SRX008254
##      3      3      3      3      3      3      3      3
## SRX012270 SRX008012 SRX008170 SRX008263 SRX008268 SRX016332 SRX008028 SRX008191
##      3      2      2      2      2      2      2      2
## SRX008223 SRX008260 SRX008021 SRX008203 SRX008226 SRX008229 SRX008232 SRX012271
##      2      2      2      2      2      2      2      2
## SRX008171 SRX008186 SRX008240 SRX010758 SRX016331 SRX008155 SRX008169 SRX008172
##      1      1      1      1      1      2      2      2
## SRX008231 SRX008542 SRX008017 SRX008209 SRX008230 SRX008270 SRX008157 SRX008204
##      2      2      2      2      2      2      1      1
## SRX008269 SRX008272 SRX008276
##      1      1      1
##
## Within cluster sum of squares by cluster:
## [1] 0.12051389 0.16631806 0.04478385
## (between_SS / total_SS = 83.6 %)
##
## Available components:
##
## [1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
## [6] "betweenss"    "size"         "iter"         "ifault"
```

6. Utiliza la función `fviz_cluster()` para hacer una representación gráfica del resultado del punto 5. El primer argumento es el objeto `kmeans` creado en el punto 5. También hay que pasarle el `data.frame` con los datos que se han analizado en el argumento `data`. Podéis utilizar otros argumentos para personalizar el gráfico (consultar la ayuda).

Solución:

```
fviz_cluster(kmean.clus, data = main.components,
  palette = RColorBrewer::brewer.pal(3, "Set2"),
  ggtheme = theme_minimal(),
  main = "k-Means Sample Clustering")
```

k-Means Sample Clustering



Identificar las variables más importantes en un conjunto de datos

Suele ser bastante difícil manejar conjuntos de datos en los que hay muchas variables. Una estrategia es utilizar sólo aquellas variables más informativas del conjunto. En esta sección vamos a ver dos formas de llevar a cabo esta selección: utilizando *random forest* y utilizando el análisis de componentes principales.

Identificar las variables más informativas con *random forest*

Hemos visto como utilizar *random forest* para predecir clases. Identificar las variables más informativas utilizando este algoritmo sólo requiere algunos cambios menores.

Utilizamos el paquete `randomForest` disponible en CRAN y los datos `iris` ya utilizados en el apartado anterior.

```
if(!("randomForest" %in% installed.packages()))
  install.packages("randomForest")
library("randomForest")

data(iris)
head(iris)
```

```
## Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1 5.1 3.5 1.4 0.2 setosa
## 2 4.9 3.0 1.4 0.2 setosa
## 3 4.7 3.2 1.3 0.2 setosa
## 4 4.6 3.1 1.5 0.2 setosa
## 5 5.0 3.6 1.4 0.2 setosa
## 6 5.4 3.9 1.7 0.4 setosa
```

1. Utiliza la función `sample()` para dividir la muestra en los dos grupos (*train* y *test set*) definidos anteriormente.

Solución:

```
set.seed(123)
train.rows <- sample(nrow(iris), 0.8 * nrow(iris), replace = FALSE)
train.set <- iris[train.rows, ]
test.set <- iris[-train.rows, ]
```

2. Utiliza la función `randomForest()` para construir el modelo con el *train set*.

- El primer argumento de esta función es una fórmula del tipo `Species ~ .`. En la parte izquierda se indica la variable que clasifica a los individuos. En la parte derecha debemos poner las variables utilizadas para hacer la predicción. El `.` indica que todas las variables salvo `Species`.
- El segundo argumento, `data`, es es `data.frame` en el que se basa el ajuste.
- Para obtener la importancia de cada variable utilizamos el argumento `importance=TRUE`.

Solución:

```
model <- randomForest(Species ~ ., data = train.set, importance=TRUE)
model
```

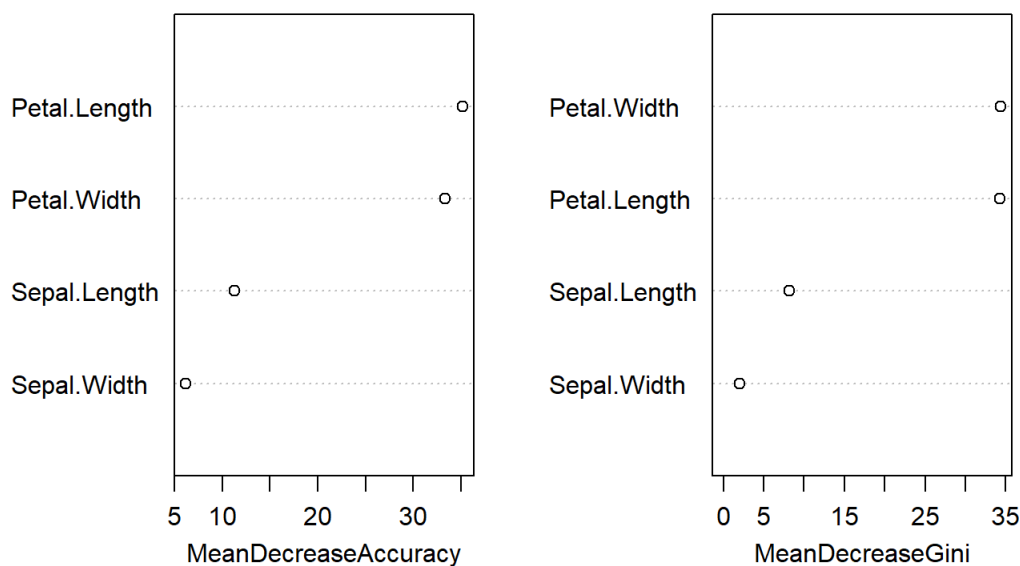
```
##
## Call:
## randomForest(formula = Species ~ ., data = train.set, importance = TRUE)
##      Type of random forest: classification
##      Number of trees: 500
## No. of variables tried at each split: 2
##
##      OOB estimate of  error rate: 5%
## Confusion matrix:
##      setosa versicolor virginica class.error
## setosa      40         0         0 0.00000000
## versicolor   0        32         3 0.08571429
## virginica    0         3        42 0.06666667
```

3. Utiliza la función `varImpPlot()` para contruir un gráfico que resuma la importancia de cada variable. Esta función requiere como argumento el modelo ajustado con *random forest* del apartado anterior. Trata de interpretar el gráfico.

Solución:

```
varImpPlot(model)
```

model



En estos gráficos se representa el efecto de no considerar las variables correspondientes. Podemos ver que las variables `Petal.Length` y `Petal.Width` son las que producen un mayor descenso tanto en la precisión del modelo como en el índice de Gini, por lo que estas dos variables serán las más importantes.

Identificar las variables más informativas con el análisis de componentes principales

Hemos visto el análisis de componentes principales (PCA) como una técnica de reducción de dimensiones: un método para reducir el tamaño de nuestros conjuntos de datos manteniendo la máxima información posible. A partir de PCA podemos hacernos una idea de que variables en el conjunto de datos original contribuyen más a reducir la dimensión y, en consecuencia, son las más relevantes.

Utilizamos el paquete `factoextra` disponible en CRAN y los datos `iris` ya utilizados en el apartado anterior.

```
if(!("factoextra" %in% installed.packages()))
  install.packages("factoextra")
library("factoextra")

data(iris)
head(iris)
```

```
## Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1      5.1      3.5      1.4      0.2 setosa
## 2      4.9      3.0      1.4      0.2 setosa
## 3      4.7      3.2      1.3      0.2 setosa
## 4      4.6      3.1      1.5      0.2 setosa
## 5      5.0      3.6      1.4      0.2 setosa
## 6      5.4      3.9      1.7      0.4 setosa
```

1. Realiza el PCA con las cuatro variables cuantitativas de `iris` utilizando la función genérica `prcomp()` . Los datos deben estar estandarizados luego debes utilizar los parámetros `scale = TRUE` y `center = TRUE` .

Solución:

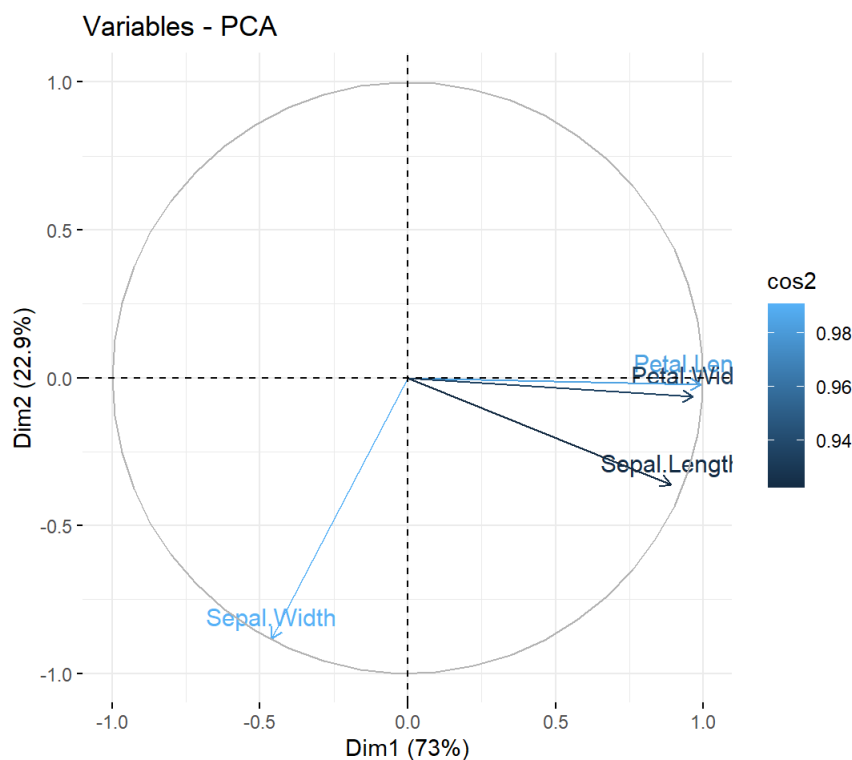
```
pca.result <- prcomp(iris[,1:4], scale=TRUE, center=TRUE)
summary(pca.result)
```

```
## Importance of components:
##              PC1  PC2  PC3  PC4
## Standard deviation  1.7084 0.9560 0.38309 0.14393
## Proportion of Variance 0.7296 0.2285 0.03669 0.00518
## Cumulative Proportion 0.7296 0.9581 0.99482 1.00000
```

2. Utiliza la función `fviz_pca_var()` para representar el resultado. Esta función requiere como primer argumento el resultado del PCA realizado en 1. Además utilizar el argumento `col.var="cos2"` para representar la importancia de las variables en un círculo con una escala de color. Trata de interpretar el resultado.

Solución:

```
fviz_pca_var(pca.result, col.var="cos2")
```



En este gráfico observamos flechas que representan cada variable del conjunto de datos original. Cuando más cerca estén dos variables mayor correlación entre ellas. El color indica la cantidad " $\cos^2$ " que es una medida de la contribución de cada variable, mayor contribución a mayor valor de " $\cos^2$ ". En esta caso las variables más importantes serían `Petal.Length` y `Sepal.Width` . Las variables `Petal.Length` y `Petal.Width` están muy correlacionadas por lo que sólo se identificaría una de ellas como relevante. Notad que este resultado es diferente al obtenido con el análisis